

Social Engine Recovery

Round 1 - Phase 2: SQL Analytics

Data Vortex :: AARUUSH'26

Team: SPARTANS

Date: 15 September 2026

This document contains SQL solutions for all 16 analytical challenges from Phase 2 of the Data Vortex hackathon. Each solution includes the SQL query, execution output, and a detailed logic explanation. All queries were executed against a SQLite database containing the cleaned Social Engine dataset.

Schema Design & Table Structuring

Database Engine: SQLite 3

Normalized relational schema with two primary tables and one auxiliary table:

TABLE: users

user_id	TEXT PRIMARY KEY
location	TEXT NOT NULL
language	TEXT NOT NULL
account_created	TEXT NOT NULL
follower_count	INTEGER NOT NULL

TABLE: posts (cleaned)

post_id	TEXT PRIMARY KEY
user_id	TEXT FK -> users.user_id
platform	TEXT NULLABLE (missing = corruption)
text_content	TEXT NULLABLE (missing = corruption)
timestamp	TEXT NOT NULL
likes	REAL NULLABLE (missing/negative = cleaned)
shares	INTEGER NOT NULL
comments	INTEGER NOT NULL

TABLE: posts_corrupted (for H5 anomaly detection)

Same columns as posts, all TEXT type
Contains raw uncleaned data

INDEXES:

idx_posts_user_id	ON posts(user_id)
idx_posts_platform	ON posts(platform)
idx_users_follower_count	ON users(follower_count)

Design Decisions:

1. NULLABLE columns preserve data provenance from corruption
2. COALESCE(likes,0) handles NULLs in aggregations
3. Separate corrupted table for H5 without mixing clean data
4. Foreign key enforces referential integrity

E1 - Platform Popularity

Logic Explanation:

Filter out NULL platforms, GROUP BY platform, COUNT posts, sort descending.

SQL Query:

```
SELECT platform,
       COUNT(*) AS num_posts
FROM posts
WHERE platform IS NOT NULL
GROUP BY platform
ORDER BY num_posts DESC;
```

Output (5 rows):

platform	num_posts
Facebook	2074
YouTube	2073
Twitter	2049
Reddit	2031
Instagram	1989

E2 - Most Engaged Posts (Top 10)

Logic Explanation:

Calculate total_engagement = likes + shares + comments. Filter out posts with NULL likes. Order descending, LIMIT 10.

SQL Query:

```
SELECT post_id,
       platform,
       likes,
       shares,
       comments,
       (likes + shares + comments) AS total_engagement
FROM posts
WHERE likes IS NOT NULL
ORDER BY total_engagement DESC
LIMIT 10;
```

Output (10 rows):

post_id	platform	likes	shares	comments	total_engagement
ycjj5zzt7mvx	Instagram	4983.0	1919	991	7893.0
wo7py9aljg3t	Reddit	4864.0	1981	948	7793.0
gmoeib832zbs	Facebook	4902.0	1880	982	7764.0
5kvuyvf38nqx	YouTube	4923.0	1971	861	7755.0
pvf13d8hj7jd	Instagram	4989.0	1840	909	7738.0
tdgjjylpua20	nan	4979.0	1932	812	7723.0
tne7s3o4l4wd	Instagram	4931.0	1903	878	7712.0
a1kiwl618kzy	Facebook	4811.0	1952	920	7683.0
fp89q1ickn9w	Twitter	4740.0	1933	955	7628.0
5n161ir5hhhr	YouTube	4751.0	1981	878	7610.0

E3 - Average Engagement by Platform

Logic Explanation:

GROUP BY platform, compute AVG for each engagement metric. Sum the averages for total. YouTube leads with 4044.54.

SQL Query:

```
SELECT platform,
       ROUND(AVG(likes), 2) AS avg_likes,
       ROUND(AVG(shares), 2) AS avg_shares,
       ROUND(AVG(comments), 2) AS avg_comments,
       ROUND(AVG(likes) + AVG(shares) + AVG(comments), 2) AS avg_total_engagement
FROM posts
WHERE platform IS NOT NULL
GROUP BY platform
ORDER BY avg_total_engagement DESC;
```

Output (5 rows):

platform	avg_likes	avg_shares	avg_comments	avg_total_engageme
YouTube	2528.33	1011.83	504.38	4044.54
Instagram	2494.51	1040.84	499.8	4035.15
Facebook	2526.55	984.17	506.94	4017.66
Reddit	2487.8	1002.23	511.18	4001.2
Twitter	2439.87	1005.39	506.13	3951.39

E4 - Highly Shared but Poorly Liked

Logic Explanation:

Simple WHERE clause filtering: shares > 1500 AND likes < 500. These posts have abnormally high sharing relative to their like count.

SQL Query:

```
SELECT post_id,
       platform,
       likes,
       shares,
       comments
FROM posts
WHERE shares > 1500
      AND likes < 500
ORDER BY shares DESC;
```

Output (241 rows):

post_id	platform	likes	shares	comments
euvr0r10wrj6	Facebook	453.0	2000	408
oiszojqm6qnn	Instagram	390.0	1999	858
f2e5kdfldedz	nan	447.0	1997	641
mgv7p46wzpek	Reddit	252.0	1993	971
u1aa801qvxeu	nan	162.0	1992	306
6s9mxoemn488	Twitter	169.0	1991	856
0nsga7zrxpvt	YouTube	390.0	1991	848
lit2hyqg0v0l	Facebook	14.0	1990	769
0xytg6ityrle	Instagram	322.0	1989	170
twgx52qb72eo	YouTube	462.0	1989	636
k8hwwxunhdf4	YouTube	25.0	1988	615
e9f22k15r0e4	nan	449.0	1986	132
vlh7lbn658qt	Twitter	357.0	1982	312
elh8jw3o1btq	Twitter	430.0	1981	209
nb0mfumbhk9s	Instagram	16.0	1979	534
ajp2c wd4xbzh	YouTube	328.0	1978	302
p1c3eh76z3c6	nan	135.0	1972	421
ch6cl3ai3vh5	YouTube	229.0	1971	664
t44gzf69bxbh4	Instagram	87.0	1970	656
bnxenpej5or3	Facebook	33.0	1968	823
vc18uw65wql1	Reddit	489.0	1968	920
9uo64r98h0cj	Twitter	316.0	1967	804
tqk2zo258e5g	Reddit	425.0	1967	164
27w2t0eudwzl	Twitter	6.0	1966	340
t546wl31j3w5	nan	432.0	1965	974

... and 216 more rows (truncated for readability)

E5 - Users With Large Audiences (>40K followers)

Logic Explanation:

Direct filter on follower_count > 40000, sorted descending.

SQL Query:

```
SELECT user_id,
       location,
       language,
       follower_count
FROM users
WHERE follower_count > 40000
ORDER BY follower_count DESC;
```

Output (293 rows):

user_id	location	language	follower_count
user_3o7w6602	Berlin, Germany	ja	49944
user_u98jwp3f	Chicago, USA	zh	49936
user_usts5yuo	Shanghai, China	ja	49933
user_d4eat3v3	Tokyo, Japan	de	49914
user_siuvpkza	Chicago, USA	en	49905
user_ujlljin7	Chicago, USA	es	49855
user_kr0ydzhh	Barcelona, Spain	fr	49836
user_9asov0m8	Singapore	hi	49763
user_84k7x6zn	Barcelona, Spain	ja	49736
user_t028mbub	Osaka, Japan	en	49727
user_i8ncp2ai	Delhi, India	fr	49722
user_sjm4thcl	Dubai, UAE	en	49721
user_b8ysn8r5	Shanghai, China	hi	49699
user_kf84zwv2	Paris, France	es	49575
user_8i81i0p7	Los Angeles, USA	en	49518
user_ixw5ddo	Chicago, USA	de	49484
user_reqsqicb	Toronto, Canada	hi	49432
user_5wmdl0y4	Dubai, UAE	hi	49404
user_9ksxfq3r	Manchester, UK	zh	49368
user_mtuzxdlb	Melbourne, Australia	fr	49341
user_wfoibt5f	Melbourne, Australia	pt	49332
user_t3vc5gvq	Berlin, Germany	pt	49312
user_2kmgah02	Houston, USA	en	49291
user_ul3t78j5	Manchester, UK	ar	49287
user_al9p1gnu	Munich, Germany	pt	49268

... and 268 more rows (truncated for readability)

M1 - Which Locations Generate the Most Engagement?

Logic Explanation:

JOIN posts with users on user_id. COALESCE(likes, 0) handles NULLs. GROUP BY location, SUM engagement, rank descending. Los Angeles leads.

SQL Query:

```
SELECT u.location,
       COUNT(p.post_id) AS num_posts,
       ROUND(SUM(COALESCE(p.likes, 0) + p.shares + p.comments), 0) AS total_engagement
FROM posts p
JOIN users u ON p.user_id = u.user_id
GROUP BY u.location
ORDER BY total_engagement DESC;
```

Output (33 rows):

location	num_posts	total_engagement
Los Angeles, USA	459	1637056.0
Munich, Germany	452	1621139.0
Shanghai, China	451	1589433.0
Barcelona, Spain	439	1575363.0
Dubai, UAE	421	1501224.0
Melbourne, Australia	421	1473055.0
Houston, USA	422	1440417.0
Rio de Janeiro, Braz	414	1439636.0
Mumbai, India	413	1430991.0
Osaka, Japan	398	1418193.0
Milan, Italy	403	1386887.0
London, UK	386	1383518.0
Chicago, USA	392	1361516.0
Johannesburg, South	383	1347497.0
Paris, France	379	1337961.0
New York, USA	372	1332793.0
Beijing, China	373	1330085.0
Sao Paulo, Brazil	376	1318070.0
Singapore	358	1273655.0
Berlin, Germany	355	1271805.0
Tokyo, Japan	354	1250973.0
Toronto, Canada	350	1238527.0
Manchester, UK	331	1169666.0
Vancouver, Canada	331	1158078.0
Rome, Italy	338	1136693.0

... and 8 more rows (truncated for readability)

M2 - Do High Follower Users Get More Engagement?

Logic Explanation:

Use CASE expression to bucket users into High/Low groups. JOIN with posts, calculate AVG engagement per post per group. Surprisingly, low-follower users get slightly more engagement.

SQL Query:

```
SELECT
    CASE
        WHEN u.follower_count >= 25000 THEN 'High (>=25K)'
        ELSE 'Low (<25K)'
    END AS follower_group,
    ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement_per_post
FROM posts p
JOIN users u ON p.user_id = u.user_id
GROUP BY follower_group
ORDER BY avg_engagement_per_post DESC;
```

Output (2 rows):

follower_group	avg_engagement_per
Low (<25K)	3535.84
High (>=25K)	3508.45

M3 - Most Active Users (Top 10)

Logic Explanation:

JOIN posts with users. GROUP BY user_id, COUNT posts, SUM engagement. Order by post_count DESC to find the most prolific posters.

SQL Query:

```
SELECT p.user_id,
       u.follower_count,
       u.location,
       COUNT(p.post_id) AS post_count,
       ROUND(SUM(COALESCE(p.likes, 0) + p.shares + p.comments), 0) AS total_engagement
FROM posts p
JOIN users u ON p.user_id = u.user_id
GROUP BY p.user_id
ORDER BY post_count DESC, total_engagement DESC
LIMIT 10;
```

Output (10 rows):

user_id	follower_count	location	post_count	total_engagement
user_zqv2vrf5	13531	Vancouver, Canada	22	83224.0
user_nfo3ih5u	40429	Barcelona, Spain	19	69650.0
user_q3vs0uj7	32714	Manchester, UK	18	56398.0
user_8wk0j5ae	36943	Los Angeles, USA	18	53069.0
user_n0ok02rt	2531	Dubai, UAE	18	51525.0
user_0irp4abu	44206	Toronto, Canada	17	59182.0
user_ujllj1n7	49855	Chicago, USA	16	63275.0
user_i1e1kek5	46765	Vancouver, Canada	16	62400.0
user_uerv85na	1824	Rome, Italy	16	60629.0
user_xetn4exw	14902	Dubai, UAE	16	59677.0

M4 - Platform Behaviour by High Follower Users (>=30K)

Logic Explanation:

Filter users with >= 30K followers, exclude NULL platforms. GROUP BY platform, compute AVG engagement. Instagram performs best for high-follower users.

SQL Query:

```
SELECT p.platform,  
       ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement_per_post  
FROM posts p  
JOIN users u ON p.user_id = u.user_id  
WHERE u.follower_count >= 30000  
      AND p.platform IS NOT NULL  
GROUP BY p.platform  
ORDER BY avg_engagement_per_post DESC;
```

Output (5 rows):

platform	avg_engagement_per
Instagram	3622.78
Reddit	3523.9
Twitter	3491.04
YouTube	3488.44
Facebook	3441.39

M5 - Detect Suspicious Engagement (Top 20)

Logic Explanation:

Suspicious = shares > (likes + comments). This indicates artificial amplification or bot-driven sharing behaviour. 2,915 such posts exist.

SQL Query:

```
SELECT post_id,
       platform,
       likes,
       shares,
       comments,
       (COALESCE(likes, 0) + shares + comments) AS total_engagement
FROM posts
WHERE shares > (COALESCE(likes, 0) + comments)
ORDER BY total_engagement DESC
LIMIT 20;
```

Output (20 rows):

post_id	platform	likes	shares	comments	total_engagement
rryxmp0nra55	Instagram	981.0	1997	958	3936.0
kyqf31ges2c6	Reddit	1091.0	1988	846	3925.0
ph0q0420bulu	Reddit	999.0	1974	928	3901.0
46yninbfki54	Reddit	1836.0	1952	96	3884.0
am8fodsgv86c	Reddit	990.0	1981	904	3875.0
j9j0ys6a50me	Reddit	961.0	1959	952	3872.0
x6c2hubogvyx	Facebook	1062.0	1971	833	3866.0
hzofcqc9jycj	Facebook	1496.0	1933	435	3864.0
2xcg9ld7du67	Twitter	1425.0	1999	440	3864.0
zv4ja8bp4xx	nan	1503.0	1997	338	3838.0
ji1jnvk4k58wl	YouTube	1784.0	1951	79	3814.0
oeooi7x38tvd	YouTube	1724.0	1967	110	3801.0
grj9ylw686o0	Instagram	1003.0	1959	838	3800.0
uakcg8xdn7p6	Twitter	1608.0	1980	210	3798.0
xpizhsi6g2mf	Instagram	1253.0	1954	591	3798.0
l6yzpko24igw	Instagram	867.0	1938	975	3780.0
bqml2pvfs2ou	YouTube	1387.0	1981	381	3749.0
1uhtybbddber	YouTube	1531.0	1880	338	3749.0
qq86lkjrfzlt	YouTube	897.0	1999	850	3746.0
fw58tjh465dd	Facebook	802.0	1959	980	3741.0

H1 - Users With Abnormally High Engagement

Logic Explanation:

CTE 1: Calculate per-user average engagement. CTE 2: Calculate overall average across all posts. Filter users whose avg > 2x overall. Result: 0 users exceed threshold (7044.63) - engagement is uniformly distributed in this dataset.

SQL Query:

```
WITH user_engagement AS (  
    SELECT p.user_id,  
           u.location,  
           u.follower_count,  
           COUNT(p.post_id) AS post_count,  
           ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement  
    FROM posts p  
    JOIN users u ON p.user_id = u.user_id  
    GROUP BY p.user_id  
,  
overall AS (  
    SELECT AVG(COALESCE(likes, 0) + shares + comments) AS overall_avg  
    FROM posts  
)  
SELECT ue.user_id,  
       ue.location,  
       ue.follower_count,  
       ue.post_count,  
       ue.avg_engagement,  
       ROUND(o.overall_avg, 2) AS overall_avg,  
       ROUND(2 * o.overall_avg, 2) AS threshold  
FROM user_engagement ue, overall o  
WHERE ue.avg_engagement > 2 * o.overall_avg  
ORDER BY ue.avg_engagement DESC;
```

Output (0 rows):

No results found (empty result set).

Analysis: No individual user exceeds 2x the overall average engagement per post (threshold: 7044.63). This indicates engagement is uniformly distributed across users, which is unusual for real social media data.

H2 - Rank Users Within Their Location (Top 3)

Logic Explanation:

Window function RANK() OVER (PARTITION BY location ORDER BY engagement DESC) assigns ranks within each location. Filter WHERE rank <= 3 to get top 3 per city.

SQL Query:

```
WITH user_location_engagement AS (
    SELECT p.user_id,
           u.location,
           SUM(COALESCE(p.likes, 0) + p.shares + p.comments) AS total_engagement,
           RANK() OVER (
               PARTITION BY u.location
               ORDER BY SUM(COALESCE(p.likes, 0) + p.shares + p.comments) DESC
           ) AS location_rank
    FROM posts p
    JOIN users u ON p.user_id = u.user_id
    GROUP BY p.user_id, u.location
)
SELECT user_id,
       location,
       ROUND(total_engagement, 0) AS total_engagement,
       location_rank
FROM user_location_engagement
WHERE location_rank <= 3
ORDER BY location, location_rank;
```

Output (99 rows):

user_id	location	total_engagement	location_rank
user_nfo3ih5u	Barcelona, Spain	69650.0	1
user_24wzfb8b	Barcelona, Spain	53039.0	2
user_5s9ifp0y	Barcelona, Spain	53011.0	3
user_cdzp4vm2	Beijing, China	48019.0	1
user_gre0h06x	Beijing, China	45311.0	2
user_ttlouv1q	Beijing, China	44933.0	3
user_m5z8a3sg	Berlin, Germany	53951.0	1
user_6e99lerx	Berlin, Germany	52526.0	2
user_rtp2dykx	Berlin, Germany	47867.0	3
user_cuzig14g	Cairo, Egypt	52541.0	1
user_g37hrbp3	Cairo, Egypt	51740.0	2
user_g634p4h2	Cairo, Egypt	43570.0	3
user_ujllj1n7	Chicago, USA	63275.0	1
user_pr7bcf45	Chicago, USA	57393.0	2
user_u98jwp3f	Chicago, USA	48996.0	3
user_zq2nib62	Delhi, India	50413.0	1
user_8776jund	Delhi, India	48780.0	2
user_yu2w96i5	Delhi, India	45948.0	3
user_xetn4exw	Dubai, UAE	59677.0	1
user_rsrg2sgy	Dubai, UAE	54505.0	2
user_ctdjnqvi	Dubai, UAE	53465.0	3
user_68enpikx	Houston, USA	55231.0	1
user_r7eg1rac	Houston, USA	51916.0	2
user_44h4xl31	Houston, USA	43545.0	3
user_9mtets0p	Johannesburg, South	58081.0	1

... and 74 more rows (truncated for readability)

H3 - Exceptional Posts (>=2x Platform Average)

Logic Explanation:

CTE computes each platform's average engagement. Main query JOINS back and filters posts where engagement >= 2x their platform's average. These are statistically exceptional outliers.

SQL Query:

```
WITH platform_avg AS (
    SELECT platform,
           AVG(COALESCE(likes, 0) + shares + comments) AS avg_engagement
    FROM posts
    WHERE platform IS NOT NULL
    GROUP BY platform
)
SELECT p.post_id,
       p.platform,
       ROUND(COALESCE(p.likes, 0) + p.shares + p.comments, 0) AS post_engagement,
       ROUND(pa.avg_engagement, 2) AS platform_avg_engagement
FROM posts p
JOIN platform_avg pa ON p.platform = pa.platform
WHERE (COALESCE(p.likes, 0) + p.shares + p.comments) >= 2 * pa.avg_engagement
ORDER BY post_engagement DESC
LIMIT 20;
```

Output (20 rows):

post_id	platform	post_engagement	platform_avg_engag
ycjj5zzt7mvx	Instagram	7893.0	3543.53
wo7py9aljg3t	Reddit	7793.0	3550.43
gmoeib832zbs	Facebook	7764.0	3543.78
5kvuyvf38nqx	YouTube	7755.0	3545.7
pvfl3d8hj7jd	Instagram	7738.0	3543.53
tne7s3o4l4wd	Instagram	7712.0	3543.53
a1kiwl618kzy	Facebook	7683.0	3543.78
fp89q1ickn9w	Twitter	7628.0	3454.84
5n161ir5hhhr	YouTube	7610.0	3545.7
g7d0tgdwipoy	Reddit	7583.0	3550.43
rh7jt9kj7tw8	YouTube	7581.0	3545.7
za6nw486cyn2	YouTube	7540.0	3545.7
d7a3bjdzzi5w	Reddit	7500.0	3550.43
xzt681si3ibm	Reddit	7479.0	3550.43
k5qkhdj5zpnw	Facebook	7477.0	3543.78
2ubp2yyrtncq	YouTube	7472.0	3545.7
mb370uy6ajb1	Instagram	7468.0	3543.53
5dsm9m39dfj3	Facebook	7455.0	3543.78
s7wyr8sbp28r	YouTube	7444.0	3545.7
sohx1tia2np4	Twitter	7440.0	3454.84

H4 - Follower to Engagement Anomaly

Logic Explanation:

Multi-level analysis: CTE 1 aggregates per-user totals. CTE 2 calculates the 90th percentile threshold using OFFSET. Filter: < 5K followers BUT engagement in top 10%. These are anomalously high-performing low-follower accounts.

SQL Query:

```
WITH user_totals AS (
    SELECT p.user_id,
           u.follower_count,
           u.location,
           SUM(COALESCE(p.likes, 0) + p.shares + p.comments) AS total_engagement,
           COUNT(p.post_id) AS post_count
    FROM posts p
    JOIN users u ON p.user_id = u.user_id
    GROUP BY p.user_id
),
percentile_threshold AS (
    SELECT total_engagement AS threshold
    FROM user_totals
    ORDER BY total_engagement DESC
    LIMIT 1 OFFSET (SELECT CAST(COUNT(*) * 0.10 AS INTEGER) FROM user_totals)
)
SELECT ut.user_id,
       ut.follower_count,
       ut.location,
       ut.post_count,
       ROUND(ut.total_engagement, 0) AS total_engagement
FROM user_totals ut, percentile_threshold pt
WHERE ut.follower_count < 5000
      AND ut.total_engagement >= pt.threshold
ORDER BY ut.total_engagement DESC;
```

Output (15 rows):

user_id	follower_count	location	post_count	total_engagement
user_uerv85na	1824	Rome, Italy	16	60629.0
user_hdas0iau	1620	Rio de Janeiro, Braz	16	59073.0
user_fgjkkrie	2211	Lyon, France	14	59064.0
user_r7egirac	2052	Houston, USA	12	51916.0
user_n0ok02rt	2531	Dubai, UAE	18	51525.0
user_6wra58f7	4953	Johannesburg, South	14	50515.0
user_5oe5t3js	3151	London, UK	13	50383.0
user_67hyf45u	898	Vancouver, Canada	11	49660.0
user_rr1uzkql	1069	Milan, Italy	14	45579.0
user_cqaitbuv	3382	Johannesburg, South	11	45510.0
user_06v0exkg	552	Barcelona, Spain	12	45239.0
user_knjtsbf3	1298	Toronto, Canada	11	44045.0
user_zozhwbis	4580	Paris, France	11	43697.0
user_lfwnie3f	2127	Shanghai, China	14	43691.0
user_0aacyvtz	1452	Rome, Italy	11	43384.0

H5 - Identify Data Anomalies (Corrupted Dataset)

Logic Explanation:

Uses the CORRUPTED dataset (not cleaned). CASE expression classifies each anomaly type. Checks 4 corruption patterns: negative likes (CAST to REAL), missing platform, missing text, HTML entities/tags via LIKE patterns.

SQL Query:

```
SELECT post_id,
       CASE
         WHEN (CAST(likes AS REAL) < 0) AND (platform IS NULL OR platform = '')
              AND (text_content IS NULL OR text_content = '')
              AND (text_content LIKE '%&%' OR text_content LIKE '%<div>%' OR text_content LIKE '%<br>%')
         THEN 'Negative likes | Missing platform | Missing text | HTML tags'
         WHEN (CAST(likes AS REAL) < 0) AND (platform IS NULL OR platform = '')
         THEN 'Negative likes | Missing platform'
         WHEN (CAST(likes AS REAL) < 0) AND (text_content LIKE '%&%' OR text_content LIKE '%<div>%' OR text_content LIKE '%<br>%')
         THEN 'Negative likes | HTML tags'
         WHEN (platform IS NULL OR platform = '') AND (text_content IS NULL OR text_content = '')
         THEN 'Missing platform | Missing text'
         WHEN CAST(likes AS REAL) < 0
         THEN 'Negative likes'
         WHEN platform IS NULL OR platform = ''
         THEN 'Missing platform'
         WHEN text_content IS NULL OR text_content = ''
         THEN 'Missing text content'
         WHEN text_content LIKE '%&%' OR text_content LIKE '%<div>%'
              OR text_content LIKE '%<br>%' OR text_content LIKE '%</div>%'
         THEN 'Contains HTML tags/entities'
         ELSE NULL
       END AS anomaly_type
FROM posts_corrupted
WHERE (CAST(likes AS REAL) < 0)
      OR (platform IS NULL OR platform = '')
      OR (text_content IS NULL OR text_content = '')
      OR (text_content LIKE '%&%' OR text_content LIKE '%<div>%'
          OR text_content LIKE '%<br>%' OR text_content LIKE '%</div>%')
ORDER BY anomaly_type;
```

Output (4274 rows):

post_id	anomaly_type
xd96337v76wx	Contains HTML tags/e
p7h083r7vums	Contains HTML tags/e
96xkquvldqrm	Contains HTML tags/e
ca41scke5hc1	Contains HTML tags/e
u085n5n0c3xw	Contains HTML tags/e
ibm43s0977z3	Contains HTML tags/e
11tqzysq0g7q	Contains HTML tags/e
wbb51kgps34m	Contains HTML tags/e
jejwhl2985wt	Contains HTML tags/e
drehaxy40ng	Contains HTML tags/e
c0ag156ja1lo	Contains HTML tags/e
qe6fzwx8qx0d	Contains HTML tags/e
hbq697fbmm31	Contains HTML tags/e
qjphn8r6reoe	Contains HTML tags/e
vov15c8f9hn4	Contains HTML tags/e
8xvn18f0iur9	Contains HTML tags/e
8d6ydbqjam6p	Contains HTML tags/e
doei4jjymwyx	Contains HTML tags/e
hyltbneepnz5	Contains HTML tags/e

post_id	anomaly_type
njxzp8kdis0	Contains HTML tags/e
9rpvawhhief	Contains HTML tags/e
rkkkg5aot86o	Contains HTML tags/e
0x8w7br3q0a4	Contains HTML tags/e
w1f3x7uhpot6	Contains HTML tags/e
9rliazbugj1e	Contains HTML tags/e

... and 4249 more rows (truncated for readability)

H6 - Most Suspicious High Impact Users

Logic Explanation:

Three-condition filter using CTEs: (1) follower_count < 10K, (2) avg engagement > overall average, (3) MAX(CASE WHEN shares > likes) = 1 checks if ANY post has more shares than likes. These users are suspiciously high-performing despite low follower counts.

SQL Query:

```
WITH user_stats AS (
    SELECT p.user_id,
           u.location,
           u.follower_count,
           COUNT(p.post_id) AS post_count,
           ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement,
           ROUND(SUM(COALESCE(p.likes, 0) + p.shares + p.comments), 0) AS total_engagement,
           MAX(CASE WHEN p.shares > COALESCE(p.likes, 0) THEN 1 ELSE 0 END) AS has_shares_gt_likes
    FROM posts p
    JOIN users u ON p.user_id = u.user_id
    GROUP BY p.user_id
),
overall AS (
    SELECT AVG(COALESCE(likes, 0) + shares + comments) AS overall_avg
    FROM posts
)
SELECT us.user_id,
       us.location,
       us.follower_count,
       us.post_count,
       us.avg_engagement,
       us.total_engagement
FROM user_stats us, overall o
WHERE us.follower_count < 10000
      AND us.avg_engagement > o.overall_avg
      AND us.has_shares_gt_likes = 1
ORDER BY us.total_engagement DESC;
```

Output (120 rows):

user_id	location	follower_count	post_count	avg_engagement	total_engagement
user_uerv85na	Rome, Italy	1824	16	3789.31	60629.0
user_hdas0iau	Rio de Janeiro, Braz	1620	16	3692.06	59073.0
user_fgjkkrie	Lyon, France	2211	14	4218.86	59064.0
user_9mtets0p	Johannesburg, South	8262	14	4148.64	58081.0
user_68enpikx	Houston, USA	9364	15	3682.07	55231.0
user_g9g7onlw	Munich, Germany	7981	13	4038.38	52499.0
user_14ibkds2	Singapore	7361	13	4018.62	52242.0
user_r7eg1rac	Houston, USA	2052	12	4326.33	51916.0
user_g37hrbp3	Cairo, Egypt	6699	11	4703.64	51740.0
user_6wra58f7	Johannesburg, South	4953	14	3608.21	50515.0
user_zq2nib62	Delhi, India	5153	12	4201.08	50413.0
user_5oe5t3js	London, UK	3151	13	3875.62	50383.0
user_67hyf45u	Vancouver, Canada	898	11	4514.55	49660.0
user_w4p8zi5g	Madrid, Spain	7103	13	3612.0	46956.0
user_acrn19py	Vancouver, Canada	5971	11	4248.09	46729.0
user_njtvacei	Dubai, UAE	8100	12	3864.5	46374.0
user_t38n155r	Berlin, Germany	6743	10	4597.2	45972.0
user_cqaitbuv	Johannesburg, South	3382	11	4137.27	45510.0
user_06v0exkg	Barcelona, Spain	552	12	3769.92	45239.0
user_xi0hyp64	Barcelona, Spain	5346	11	4106.09	45167.0
user_wq8mex2l	Madrid, Spain	5897	10	4502.6	45026.0
user_5k81svok	Milan, Italy	8124	12	3717.83	44614.0

Social Engine Recovery - Phase 2 SQL Analytics | Team SPARTANS

user_id	location	follower_count	post_count	avg_engagement	total_engagement
user_knjtsbf3	Toronto, Canada	1298	11	4004.09	44045.0
user_zozhwbis	Paris, France	4580	11	3972.45	43697.0
user_0aacyvtz	Rome, Italy	1452	11	3944.0	43384.0

... and 95 more rows (truncated for readability)

Phase 2 Insight Report

* Platform Distribution

Facebook leads narrowly with 2,074 posts, but all 5 platforms are remarkably balanced (within 4% of each other). This suggests equal platform integration.

* Engagement Paradox

YouTube generates the highest average engagement (4,044.54) despite not being the most popular platform. Video content drives more interaction per post.

* Follower Count Myth

Counter-intuitively, users with <25K followers get slightly MORE average engagement (3,535) than high-follower users (3,508).

* Geographic Leaders

Los Angeles generates the most total engagement (1.64M), followed by Munich and Shanghai.

* Suspicious Sharing

2,915 posts (24%) have shares exceeding likes + comments combined. This may indicate bot-driven amplification.

* Uniform Distribution

No user exceeds 2x the overall average engagement (threshold: 7,044). Engagement is uniformly distributed.

* Low-Follower Anomalies

14 users with <5K followers rank in the top 10% of engagement. user_uerv85na leads with 60,629 total engagement.

* Data Corruption Profile

4,274 corrupted posts: 525 negative likes, 1,846 missing platforms, 1,746 missing text, 663 HTML artifacts.

* Suspicious Users

120 users meet all suspicion criteria. user_uerv85na appears across multiple queries as a consistent anomaly.

* Instagram for Influencers

Among 30K+ follower users, Instagram delivers the highest avg engagement (3,622.78).